The Locality of Noncovalent Interactions: Machine-Learned Semilocal Corrections for DFT from Single Heavy Atoms

XX

(Dated: September 7, 2026)

In this article, we introduce an approach to improve the accuracy of density functional theory (DFT) calculations by machine-learned semilocal corrections trained on single heavy (non-hydrogen) atoms.

I. INTRODUCTION

The well-known Kohn–Sham equation [1] is

$$\Big(-\frac{1}{2}\nabla^2 + v_{\text{ext}} + v_{\text{H}} + v_{\text{xc}}\Big)\phi_i = \epsilon_i \phi_i. \tag{1}$$

Where ϕ_i is the Kohn–Sham orbital, ϵ_i is the Kohn–Sham orbital energy, v_{ext} is the external potential (such as the electrostatic potential of nuclei), v_{H} is the electrostatic potential of the electron, and v_{xc} is the exchange-correlation potential. According to the Hohenberg-Kohn theorem [2], exchange-correlation energy density e_{xc} is the unique functional of the ground-state electron density, $\rho = n_i|\phi_i|^2$, where n_i is the occupation number of the Kohn-Sham orbital. Then, the Kohn-Sham equation is solved iteratively utilizing the so-called potential term $v_{\text{xc}} = \delta E_{\text{xc}}/\delta\rho$. So, once this unique functional is obtained, the properties of the molecules can be calculated accurately and efficiently. This is the main idea of the density functional theory (DFT) [3].

One of the challenging tasks of the DFT calculation is to find the unique functional between the exact exchange-correlation energy density e_{xc} and the electron density ρ. Numerous approximations have been proposed for this functional, including early local-density approximation functionals [3], hybrid functionals [4], and the rapidly growing class of machine-learning (ML) functionals which have been developed in recent years [5–9]. However, many of those ML functionals suffer from lack of transferability or the demand of large training sets.

The most popular test set, which used as a benchmark to evaluate their accuracy and transferability, is the GMTKN55 dataset. The GMTKN55 dataset divides the benchmark into 5 different categories: reaction energies for small or large systems, reaction barrier heights, intermolecular non-covalent interactions (NCI), and intramolecular NCI. The training set of ML functionals is often a similar composition of those categories as the test set, ensuring that the model can generalize well to unseen data [7, 8]. In this article, we only use the 1 categories: reaction energies for small systems as the training set, while testing on the total GMTKN55 dataset. The reason for this choice is to show the transferability of the ML functional trained on a limited category of data.

The organization of this article is as follows. In Section 2, we introduce the neural network architecture used to learn the functional. In Section 3, we present the training procedure and the results on the GMTKN55 dataset. Finally, we conclude with a discussion on the transferability of the ML functional.

II. THE NEURAL NETWORK

We use the transformer [11] and e3nn network [**?**] as the neural network to learn the functional between the electron density and the energy density of the molecule. After the training, the neural network can predict the energy of the molecule given the electron density and then can be used to optimize the electron density iteratively using the self-consistent field (SCF) method.

The architecture of the neural network is shown in Fig. 1. The total number of parameters in the neural network is around 1.2 million. The input of the neural network is a set of the energy density $\tilde{\boldsymbol{e}}_{\text{cube}}(\mathbf{r})$ with the size of $(N, 27, 4)$, and the output is the energy density $e(\mathbf{r})$ with the size of $(N, 1)$. N is the number of the grid points in the normal dft procedure.

The input of the neural network is the set of energy densities $\tilde{\boldsymbol{e}}_{\text{cube}}(\mathbf{r})$ defined below.

$$\begin{aligned}\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}}) =& \big\{\tilde{e}(\mathbf{r}'_c) \text{ for } \mathbf{r}'_c \in \mathbf{r}_{\text{cube}}\big\} \\ =& \big\{\tilde{e}(\mathbf{r}), \tilde{e}(\mathbf{r}_{\text{aux}})\big\}.\end{aligned} \tag{2}$$

The $\tilde{e}(\mathbf{r})$ is the energy density calculated by the 4 functionals (LDA, VWN3, B88 and LYP) from the original B3LYP functional [4]. The $\mathbf{r}_{\text{cube}}$ is a cube around the grid point $\mathbf{r}$ with the size of $3 \times 3 \times 3$ and the distance between two adjacent grid points is d. This means that for each grid point $\mathbf{r}$, we involve a small region around it to capture the local environment and provide more features for the neural network. We refer to the grid points within the cube, excluding the central point, as auxiliary grid points.

The energy density at the grid point $\mathbf{r}$, denoted as $e(\mathbf{r})$, can be predicted by the neural network as

$$e_{\text{NN}}(\mathbf{r}) = \text{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}}); \mathcal{W}\big), \tag{3}$$

where $\text{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}}); \mathcal{W}\big)$ represents the neural network function parameterized by $\mathcal{W}$, taking $\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}})$ as input. The total exchange-correlation energy is obtained by integrating the energy density over the entire grid. Then the exchange-correlation potential can be calculated by the functional derivative of the exchange-correlation energy with respect to the electron density, which is used in the Kohn-Sham equation Eq. (1), so the properties of

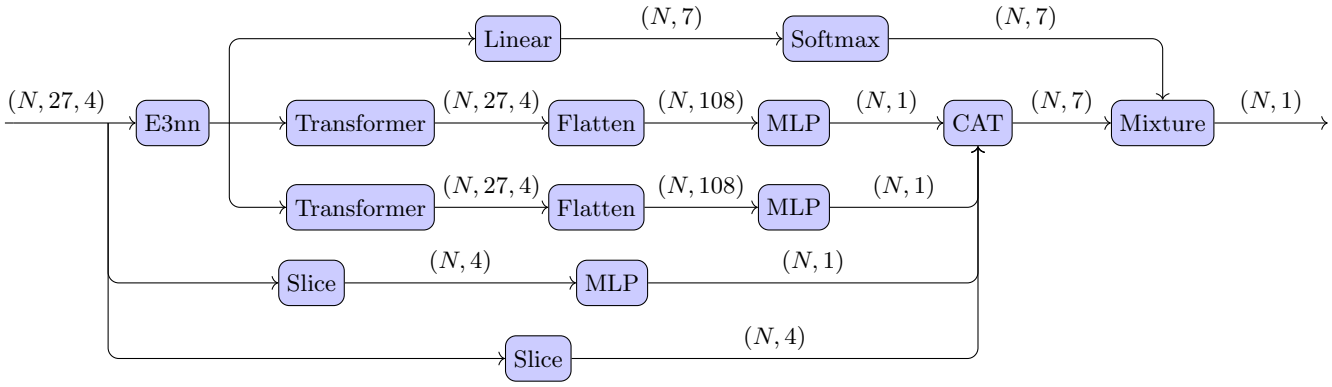

FIG. 1: The architecture of the neural network. The E3nn block [10] and the Slice block extract the invariant features from the input. The Mixture block combines the outputs from the different paths to produce the final prediction. The output of the neural network is the predicted energy density for the central grid point.

the molecules can be solved using the self-consistent field method. Details of the energy density and the potential are provided in the Appendix.

We use the E3nn block [10] and the Slice block to extract the invariant features from the input, $\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}})$. The E3nn block only use the dot production of the input features to extract the invariant features. The Transformer block we use here is only composed of multilayer of the scaled dot-product attention part of the original transformer [11]. The Mixture block combines the outputs from the different paths to produce the final prediction. Note the output, the predicted energy density for the central grid point, is only dependent on the $\tilde{\boldsymbol{e}}(\mathbf{r}_{\text{cube}})$, so we call it a semi-local ML functional. Other information about the architecture of the neural network is described in the appendix.

III. DATASET AND THE LOSS FUNCTION

A. Dataset and the loss function

The dataset we use to train the neural network is listed in the Appendix. We use 128 molecules which consistent of less than 1-2 heavy (non-hydrogen) atoms as the training set and 34 molecules contain 4-5 heavy atoms as the validation set. In this article, we use the CCSD(T)/Def2-QZVPPD [12–14] to calculate the electron density and the energy density of the molecules.

The loss function is defined as

$$\begin{aligned} L_e =& N_{\text{atom}} \frac{\left(\int \mathrm{d}\mathbf{r}_i \left(\bar{e}(\mathbf{r}_i) - e(\mathbf{r}_i)\right)\right)^2}{\left|\int \mathrm{d}\mathbf{r}_i \bar{e}(\mathbf{r}_i)\right| + \epsilon} + N_{\text{atom}} \frac{\left(\bar{E}^{\text{AE}} - E^{\text{AE}}\right)^2}{\left|\bar{E}^{\text{AE}}\right| + \epsilon} \\ &+ N_{\text{atom}} \lambda_{\text{ABS}} \int \mathrm{d}\mathbf{r}_i \left(\bar{e}(\mathbf{r}_i) - e(\mathbf{r}_i)\right)^2 + N_{\text{atom}} \left(\bar{F} - F\right)^2. \end{aligned} \tag{4}$$

Where N_{atom} is the number of atoms in the molecule, E^{AE} and $\bar{E}^{\text{AE}}$ is the atomic energy of the predicted and target values, and F and $\bar{F}$ is the force of the predicted and target values, respectively. The ϵ is a small number to avoid the division by zero and control the scale of the loss function, which is set to be 1×10^{-4} in this article. The first term is the relative error in the total energy, the second term is the error in the atomic energy, the third term is the absolute error in the total energy, and the fourth term is the error in the force; See the Appendix for the definition of the atomic energy and force.

B. Training details

The neural network is trained using the AdamW optimizer [15] with the scheduler of cosine annealing with warm restarts [16]. We set the initial learning rate to be 1×10^{-4} and then gradually decrease it to 0 using the cosine annealing scheduler, with a restart every 160 epochs and a multiplication factor of 2 for the initial learning rate after each restart. The parameters of the AdamW optimizer are set to be $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 1 \times 10^{-8}$, and the weight decay is set to be 1×10^{-12}. The weights in the loss function Eq. (**??**) are set to be $\lambda_{\text{ABS}} = 0.01$. We use a batch size of 1 and train the model for 10000 epochs, the training process is last for around 20 days. The training process is monitored using the validation set every 5 epochs, and the model with the

lowest validation loss is selected as the final model.

The detail of the training dataset and validation dataset is listed in the Appendix. The training dataset consists of 69 molecules (set A and set B) or 128 molecules (set A, set B and set C) with $1-2$ heavy (non-hydrogen) atoms, while the validation dataset consists of 115 molecules with $4-5$ heavy atoms. The validation loss is calculated using only the first two items in the loss function Eq. (4), which are the relative error in the total energy and the atomic energy error, to avoid the difficulty of calculating the absolute error and force in the total energy for large molecules.

The training is performed on a single NVIDIA A100 GPU with 80 GB memory with double precision training using PyTorch [17] and the training takes around 10 days to complete. The version of PyTorch used is 2.8.0+cu128, and CUDA version is 12.8, the version of driver is 535.247.01.

IV. RESULTS

A. Testing on the GMTKN55 Dataset

We

B. Comparison with other machine learning functionals

The main difference between our method and other machine learning functionals [5–9] is that we use the energy density as the target of the neural network instead of the total energy.

[1] W. Kohn and L. J. Sham, Self-Consistent Equations Including Exchange and Correlation Effects, Phys. Rev. **140**, A1133 (1965).

[2] P. Hohenberg and W. Kohn, Inhomogeneous Electron Gas, Phys. Rev. **136**, B864 (1964).

[3] R. G. Parr, Density Functional Theory of Atoms and Molecules, in *Horiz. Quantum Chem.*, edited by K. Fukui and B. Pullman (Springer Netherlands, Dordrecht, 1980) pp. 5–15.

[4] J. P. Perdew, J. A. Chevary, S. H. Vosko, K. A. Jackson, M. R. Pederson, D. J. Singh, and C. Fiolhais, Atoms, molecules, solids, and surfaces: Applications of the generalized gradient approximation for exchange and correlation, Phys. Rev. B **46**, 6671 (1992).

[5] R. Nagai, R. Akashi, and O. Sugino, Completing density functional theory by machine learning hidden messages from molecules, Npj Comput. Mater. **6**, 43 (2020).

[6] Y. Chen, L. Zhang, H. Wang, and W. E, DeePKS: A Comprehensive Data-Driven Approach toward Chemically Accurate Density Functional Theory, J. Chem. Theory Comput. **17**, 170 (2021).

[7] J. Kirkpatrick, B. McMorrow, D. H. P. Turban, A. L. Gaunt, J. S. Spencer, A. G. D. G. Matthews, A. Obika, L. Thiry, M. Fortunato, D. Pfau, L. R. Castellanos, S. Petersen, A. W. R. Nelson, P. Kohli, P. Mori-Sánchez, D. Hassabis, and A. J. Cohen, Pushing the frontiers of density functionals by solving the fractional electron problem, Science **374**, 1385 (2021).

[8] G. Luise, C.-W. Huang, T. Vogels, D. P. Kooi, S. Ehlert, S. Lanius, K. J. H. Giesbertz, A. Karton, D. Gunceler, M. Stanley, W. P. Bruinsma, L. Huang, X. Wei, J. G. Torres, A. Katbashev, R. C. Zavaleta, B. Máté, S.-O. Kaba, R. Sordillo, Y. Chen, D. B. Williams-Young, C. M. Bishop, J. Hermann, R. van den Berg, and P. Gori-Giorgi, Accurate and scalable exchange-correlation with deep learning (2025), arXiv:2506.14665 [physics].

[9] Z. An, J. Wang, Y. Zhang, Z. Li, J. Wu, Y. Zheng, G. Chen, and X. Zheng, Mitigating error cancellation in density functional approximations via machine learning correction, J. Chem. Phys. **163**, 054111 (2025).

[10] M. Geiger and T. Smidt, E3nn: Euclidean Neural Networks (2022), arXiv:2207.09453 [cs.LG].

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, Attention Is All You Need (2023), arXiv:1706.03762 [cs].

[12] R. J. Bartlett and M. Musiał, Coupled-cluster theory in quantum chemistry, Rev. Mod. Phys. **79**, 291 (2007).

[13] F. Weigend and R. Ahlrichs, Balanced basis sets of split valence, triple zeta valence and quadruple zeta valence quality for H to Rn: Design and assessment of accuracy, Phys. Chem. Chem. Phys. **7**, 3297 (2005).

[14] J. Zheng, X. Xu, and D. G. Truhlar, Minimally augmented Karlsruhe basis sets, Theor. Chem. Acc. **128**, 295 (2011).

[15] I. Loshchilov and F. Hutter, Decoupled Weight Decay Regularization (2019), arXiv:1711.05101 [cs].

[16] I. Loshchilov and F. Hutter, SGDR: Stochastic Gradient Descent with Warm Restarts (2017), arXiv:1608.03983 [cs].

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, PyTorch: An Imperative Style, High-Performance Deep Learning Library (2019), arXiv:1912.01703 [cs].

[18] L. Goerigk, A. Hansen, C. Bauer, S. Ehrlich, A. Najibi, and S. Grimme, A look at the density functional theory zoo with the advanced GMTKN55 database for general main group thermochemistry, kinetics and noncovalent interactions, Phys. Chem. Chem. Phys. **19**, 32184 (2017).

Appendix A: Training Details

1. Dataset

The dataset we use are listed in Table I, all the Datasets are obtained from the GMTKN55 datasets [18]. We use the molecules in datasets A, B, and C as the training set and the molecules in datasets A′ and B′ as the validation set. Datasets A, B, and C are defined as follows: dataset A comprises single-atom systems from GMTKN55 [18]; dataset B comprises molecules with one heavy (non-hydrogen) atom from W4-11; and dataset C comprises molecules with two heavy atoms from W4-11. This lead to 136 molecules in the training set and 34 molecules in the validation set, while the number of molecules in GMTKN55 datasets is 2462 [18]. To improve training efficiency and balance the dataset across different atom species, we add 9 additional copies of dataset A.

In this article, we use the electron density and energy density from CCSD(T)/Def2-QZVPPD [12–14] as the input and target of the molecules. This leads to a significant label error, which is estimated to be 2.33 kcal/mol as the mean absolute error (MAE) for the W4-11 dataset. Then all the test procedures are performed with the a smaller basis set, Def2-QZVP(D) [13, 14], which means the Def2-QZVPD basis set is used for anions and the Def2-QZVP basis set is used for other molecules, to reduce the computational cost.

We use the energy density calculated by the four functionals (LDA, VWN3, B88, and LYP) from the original B3LYP functional [4] as the input features of the neural network, not the electron density and the derivatives of the electron density directly.

$$\tilde{e}(\mathbf{r}) = \big(e_{\mathrm{LDA}}(\mathbf{r}), e_{\mathrm{VWN3}}(\mathbf{r}), e_{\mathrm{B88}}(\mathbf{r}), e_{\mathrm{LYP}}(\mathbf{r})\big). \tag{A1}$$

This choice is motivated by the very wide dynamic range of the electron density, ρ, and its derivatives, which complicates learning and can destabilize both the optimization and the validation process. By using the energy densities from established components (LDA, VWN3, B88, LYP) as input features, we provide numerically stable, physically informed representations of the local electronic environment.

The network outputs the residual energy density, defined as $e(\mathbf{r}) = e_{\mathrm{xc}}^{\mathrm{CCSD(T)}}(\mathbf{r}) - e_{\mathrm{xc}}^{\mathrm{B3LYP}}(\mathbf{r})$, where the B3LYP term is computed from the input features and the CCSD(T) term serves as the target. The total exchange-correlation energy density is then reconstructed by adding the B3LYP energy density back to the predicted residual, $E_{\mathrm{xc}}^{\mathrm{pred}} = E'_{\mathrm{xc}} + E_{\mathrm{xc}}^{\mathrm{B3LYP}}$.

Appendix B: Auxiliary Grid Points

To provide additional features for the neural network, we consider a small cube surrounding each grid point $\mathbf{r}_i = (x_i, y_i, z_i)$. This cube has dimensions of $3 \times 3 \times 3$, with d representing the distance between two adjacent grid points. To make it clear, the auxiliary grid points $\mathbf{r}_{i;\mathrm{aux}}$ are defined as all grid points within this cube, excluding the central point $\mathbf{r}_i$ itself.

$$\mathbf{r}_{i;\mathrm{aux}} = \big\{\mathbf{r}'_i \in \mathbf{r}_{i;\mathrm{cube}} \text{ and } \mathbf{r}'_i \neq \mathbf{r}_i\big\}. \tag{B1}$$

In the standard DFT procedure, each grid point is assigned a weight w_i, which is used to perform numerical integration. We set all the weights of the auxiliary grid points ($w_{i;\mathrm{aux}}$) to be zero. Thus, they do not contribute to the total energy and potential in the DFT procedure.

$$E'_{\mathrm{xc}} = \int e_{\mathrm{NN}}(\mathbf{r})\mathrm{d}\mathbf{r} = \int \mathrm{dNN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\mathrm{cube}}); \mathcal{W}\big). \tag{B2}$$

Consequently, while the auxiliary grid points influence the potential through the functional derivative, they do not directly contribute to the total energy integration. This ensures the procedure remains consistent with the standard DFT framework. The contribution of the electron density ρ to the $\mu\nu$-th element of Fock matrix which contributed by the ML exchange-correlation, $\mathrm{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}_{\mathrm{cube}}); \mathcal{W}\big)$, is calculated as

$$\begin{aligned} F_{\mu\nu}^{\mathrm{ML}} =& \frac{\delta E'_{\mathrm{xc}}}{\delta P_{\mu\nu}} \\ =& \int \mathrm{d}\mathbf{r} \sum_{\mathbf{r}' \in \mathbf{r}_{\mathrm{cube}}} \frac{\delta e_{\mathrm{NN}}(\mathbf{r}')}{\delta \rho(\mathbf{r}')} \chi_\mu(\mathbf{r}')\chi_\nu(\mathbf{r}') \\ =& \int \mathrm{d}\mathbf{r} \sum_{\mathbf{r}' \in \mathbf{r}_{\mathrm{cube}}} \frac{\delta \mathrm{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}'); \mathcal{W}\big)}{\delta \rho(\mathbf{r}')} \chi_\mu(\mathbf{r}')\chi_\nu(\mathbf{r}') \\ =& \int \mathrm{d}\mathbf{r} \sum_{\mathbf{r}' \in \mathbf{r}_{\mathrm{cube}}} \frac{\partial \mathrm{NN}\big(\tilde{\boldsymbol{e}}(\mathbf{r}'); \mathcal{W}\big)}{\partial \tilde{e}(\mathbf{r}')} \frac{\delta \tilde{e}(\mathbf{r}')}{\delta \rho(\mathbf{r}')} \chi_\mu(\mathbf{r}')\chi_\nu(\mathbf{r}'). \end{aligned} \tag{B3}$$

and the contribution of the derivative of the electron density $\nabla\rho$ to the Fock matrix is calculated similarly.

To make the neural network can capture more information about the local environment of the grid point $\mathbf{r}_i$ in the training set, we set the cube to be aligned with the main rotation axis of the secondary derivative matrix of the electron density at the grid point $\mathbf{r}_i$. This secondary derivative matrix is defined as

$$H(\mathbf{r}_i) = \begin{pmatrix} \frac{\partial^2\rho(\mathbf{r}_i)}{\partial x^2} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial x\partial y} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial x\partial z} \\ \frac{\partial^2\rho(\mathbf{r}_i)}{\partial y\partial x} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial y^2} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial y\partial z} \\ \frac{\partial^2\rho(\mathbf{r}_i)}{\partial z\partial x} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial z\partial y} & \frac{\partial^2\rho(\mathbf{r}_i)}{\partial z^2} \end{pmatrix}. \tag{B4}$$

The eigenvectors $\mathbf{u}(\mathbf{r}_i)$ of this matrix represent the main rotation axes, and we align the cube with these axes.

$$\begin{aligned} \mathbf{r}_{i;\mathrm{cube}} =& \big\{\mathbf{r}_i + d(m\mathbf{u}_1(\mathbf{r}_i) + n\mathbf{u}_2(\mathbf{r}_i) + p\mathbf{u}_3(\mathbf{r}_i)) \\ & \text{for } m, n, p \in \{-1, 0, 1\}\big\}. \end{aligned} \tag{B5}$$

Although this alignment process adds some computational cost, it provides a consistent local reference frame

TABLE I: The training and validation dataset.

	Original Dataset	Size	Number of heavy (non-hydrogen) atoms	Number of hydrogen atoms
A	GMTKN55 [1]	47	1	0
B	W4-11	22	1	$\neq 0$
C	W4-11	59	2	–
A′	GMTKN55	23	5	≤ 3
B′	GMTKN55	11	6	≤ 3

[1] Not include all atoms-species in the GMTKN55 dataset, such as Se, Ge, As, Te, I, Bi, Pb, and Sb.

for the input features, which may help the model learn more effectively. While in the test set, we do not perform this alignment process, and the cube is aligned with the global coordinate system.

Appendix C: Ablation Study

1. Ablation study on the dataset size

In this subsection, we analyze the impact of dataset size on the performance of the neural network. We train the model with varying dataset sizes and evaluate its accuracy on the validation set. The results are summarized in Table **??**.